

Debugging

Everything runs in containers; every command below is a container command.

Logs

```
make logs                # all services, followed
make logs S=api          # one service: api | worker | web | db | redis
docker compose logs --tail=200 api
```

The API logs at `LOG_LEVEL` (`DEBUG` in the dev stack). Uvicorn runs with `--reload` in dev, watching `/app/app` only, so editing a test does not restart the server.

Attaching a Python debugger

The API container has the source bind-mounted in dev, so the fastest loop is a breakpoint plus an interactive run:

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml \
  run --rm --service-ports api python -m pdb -c continue -m uvicorn
  app.main:app --host 0.0.0.0 --port 8000
```

For a REPL against the real database and the real models:

```
make shell-api
python
>>> from app.core.db import session_scope
>>> from app.models import LegoSetInstance
>>> with session_scope() as db:
...     print(db.query(LegoSetInstance).count())
```

For `debugpy`, add it to `requirements-dev.txt` and start the API with `python -m debugpy --listen 0.0.0.0:5678 -m uvicorn ...`, then publish 5678.

Frontend

The Vite dev server prints HMR errors in `make logs S=web`; runtime errors surface in the browser console and in the overlay. React Query state is easiest to inspect by adding `@tanstack/react-query-devtools` temporarily — it is deliberately not a shipped dependency.

Redis and Celery

```
docker compose exec redis redis-cli
> KEYS session:*           # active session cache entries
> KEYS ratelimit:*         # fixed-window rate-limit counters
> LLEN celery               # queued tasks
```

```
docker compose exec worker celery -A app.worker.celery_app inspect active
docker compose exec worker celery -A app.worker.celery_app inspect
registered
```

Reading what actually happened

Two tables answer almost every "why did it do that?" question.

audit_logs — who changed what, and from what to what

```
SELECT created_at, action, table_name, before, after, reason
FROM audit_logs
WHERE record_id = '<uuid>'
ORDER BY created_at;
```

before/after are JSONB snapshots of the row, so a value's entire history — for example every **current_value_eur** a LEGO set has ever had — is reconstructible without a dedicated history table.

processing_jobs — why a background task failed

```
SELECT job_type, status, attempts, last_error, payload, started_at,
completed_at
FROM processing_jobs
WHERE status IN ('FAILED', 'RETRYING')
ORDER BY created_at DESC
LIMIT 20;
```

A failure is always a row here. If a task appears to have vanished, look for its **idempotency_key**: a **SUCCEEDED** row means the retry was correctly skipped, not lost.

Common symptoms

Symptom	Cause	Fix
Error: ... has dependent containers / container name ... is	A one-shot compose run without --no-deps re-reconciles db/redis against	Always pass --no-deps to compose run and start dependencies separately (compose up -d db redis). The make/fm targets already do this

Symptom	Cause	Fix
<code>already in use</code> during a <code>run</code>	the other overlay's service definitions	
Error: no container with name ... found: no such container on first up	podman-compose unconditionally stops and removes a container before creating it	Cosmetic only — the exit code is 0 and the container starts. It disappears on subsequent runs
403 Token CSRF em falta ou inválido	The client did not echo <code>X-CSRF-Token</code> , or the token was rotated by an entity switch	Refetch <code>/api/auth/me</code> ; the session context does this automatically
403 Selecione uma entidade específica	A write was attempted with the selector on «todas»	Pick an entity — the app never guesses an owner
409 on creating a set	<code>(entity_id, set_number)</code> already exists	Add another copy instead of duplicating the model
Images render as broken	The signed URL expired (15 min)	Refetch the resource; URLs are minted per response, never cached long
<code>alembic</code> fails on startup	Two heads after a merge	<code>docker compose run --rm api alembic heads</code> , then <code>alembic merge</code>
Web container shows an empty page	<code>node_modules</code> volume predates a dependency change	<code>make down && docker volume rm finmanager_web-node-modules && make dev</code>